MODULE 3 · MACHINE LEARNING · DAY 32

Getting Data Into Shape

Scaling, building new columns, and throwing away the useless ones.



Boston University

5	14	3
sections	worked pictures	scalers, by hand

Prepared & presented by **Srinivasa Sai Chava**

WHERE WE ARE

Last time we got words in. Today we fix the numbers.

Last time	**"Dell" became three yes/no columns, and the model could finally read it.**	Now every column is a number — so we are done, surely?
The problem	**A price column runs to 80,000. A RAM column stops at 16.**	To a model, the big column shouts and the small one whispers.
Today	**Put every column on a fair footing, build better columns, and delete the ones that do not help.**	This is the step that decides whether a model works at all.

Nothing today is hard. But skip it and even a good model will quietly give you nonsense.

THE PLAN

Where we are going today

#	Section	The question it answers	Time
1	**Why scaling matters**	Why does a bigger column drown out a smaller one?	15 min
2	**The three scalers**	How do we fix it, and which one do I pick?	15 min
3	**Making new features**	Can I give the model columns it could never work out itself?	15 min
4	**Too many features**	Is more columns always better? (No.)	12 min
5	**Put it together**	In what ORDER do I do all of this?	10 min

Section 5 is the one to write down. Doing these steps in the wrong ORDER is the single most common mistake in machine learning.

SECTION 1 OF 5

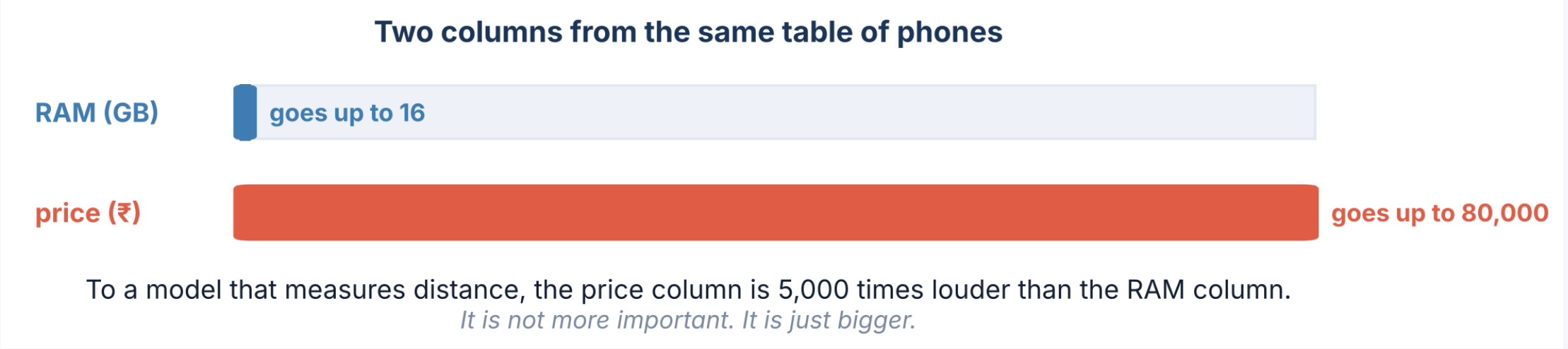
Why scaling matters

One column can be a thousand times bigger than another — and some models cannot tell the difference between big and important.

- The loudest column
- A real mistake
- Who needs scaling
- What scaling does

1.1 · THE PROBLEM

Two columns, wildly different sizes



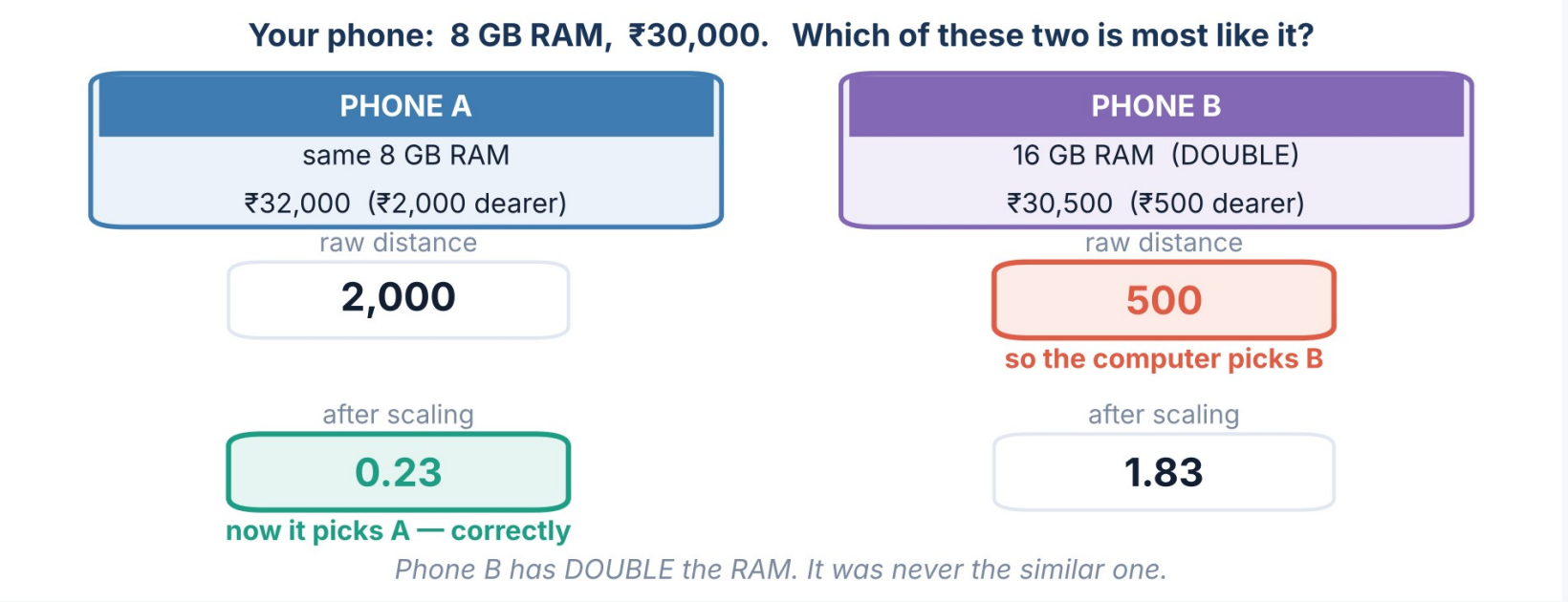
Nothing is wrong with the data. Both columns are correct. But any model that measures DISTANCE will barely notice the RAM column exists.

Why distance?

Lots of models work by asking "which other rows look like this one?" To answer that they add up how far apart the columns are — and a column measured in thousands dominates that sum completely.

1.2 · A REAL MISTAKE

Watch it pick the wrong phone



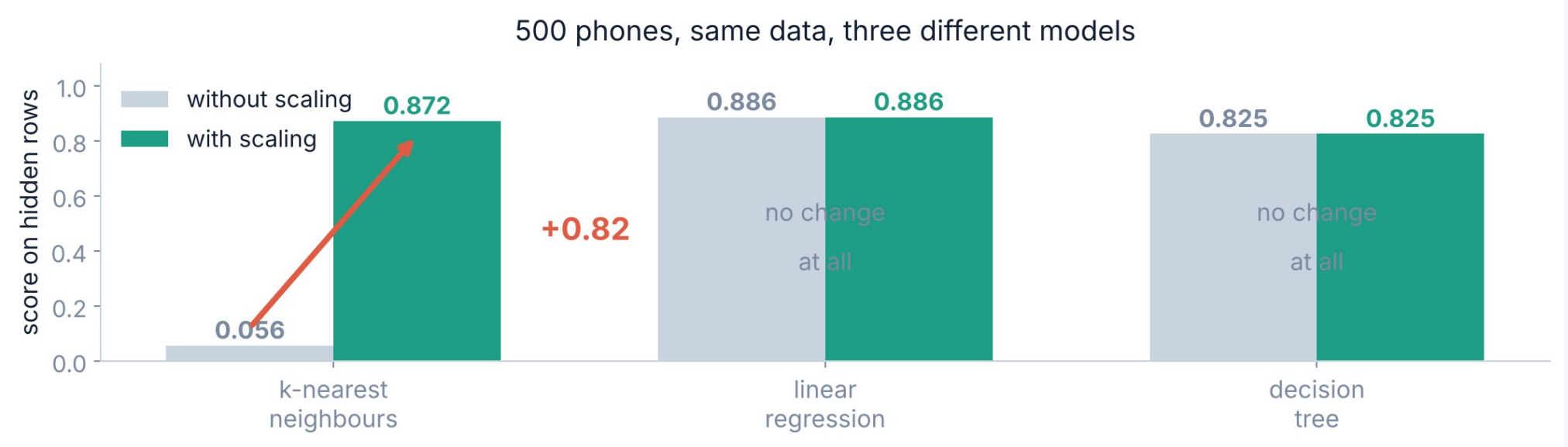
Phone B has TWICE the RAM. It was never the similar one — the price column was simply louder.

Scaling does not make your data better.
It makes every column equally LOUD.

The information is identical afterwards. Only the volume changes.

1.3 · WHO NEEDS SCALING

So who actually needs it?



YES, badly

k-nearest neighbours · SVM · k-means · PCA · neural networks

Anything that measures distance between rows.

NO, never

decision trees · random forests

A tree asks "is RAM above 8?" — bigger numbers change nothing.

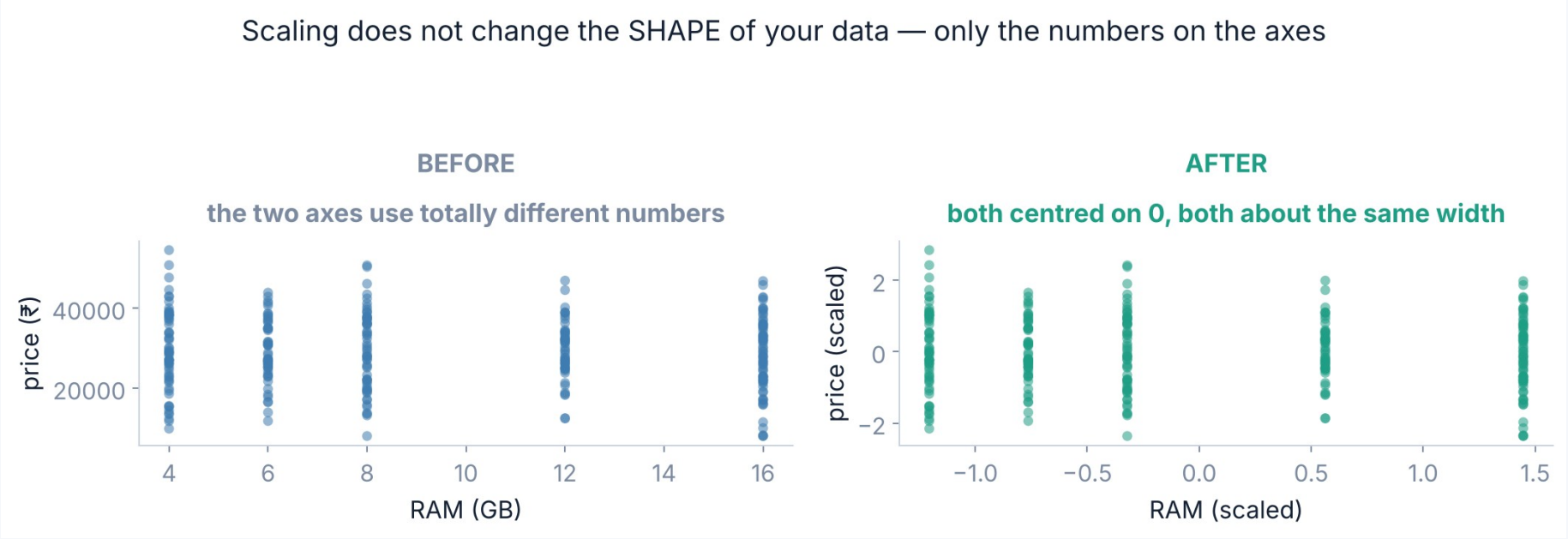
Yes, but not for accuracy

linear & logistic regression

Scale them for speed and readable weights, not for a better score.

1.4 · BEFORE AND AFTER

And what it looks like on the data



Exactly the same cloud of points. Only the numbers on the axes moved. Scaling never throws anything away.

SECTION 2 OF 5

The three scalers

Three ways to do it. Worked by hand on six salaries, so you can see exactly what each one does.

- Standard
- Min-max
- Robust
- Which to pick

2.1 · SIDE BY SIDE

Six salaries, three scalers, all worked out

Six monthly salaries (₹ thousands) — five staff and one CEO

	salary	STANDARD (x – mean) ÷ sd	MIN-MAX (x – min) ÷ range	ROBUST (x – median) ÷ IQR
	30	-0.52	+0.00	-1.00
	35	-0.48	+0.01	-0.60
	40	-0.45	+0.03	-0.20
	45	-0.41	+0.04	+0.20
	50	-0.37	+0.05	+0.60
the CEO →	400	+2.23	+1.00	+28.60

Read the last row across. Only ROBUST reports the CEO as "far out" instead of squashing everyone else.

All three keep the ORDER. All three keep the information. They differ only in what they are measured against.

2.2 · IN WORDS

What each one is really measuring

STANDARD

`(x − mean) ÷ sd`

"How many standard deviations from average is this?"

Output centred on 0, usually between −3 and +3. No upper limit.

MIN-MAX

`(x − min) ÷ range`

"Where does this sit between the smallest and largest?"

Output always between 0 and 1 — but defined by just two values.

ROBUST

`(x − median) ÷ IQR`

"How far from the typical value is this?"

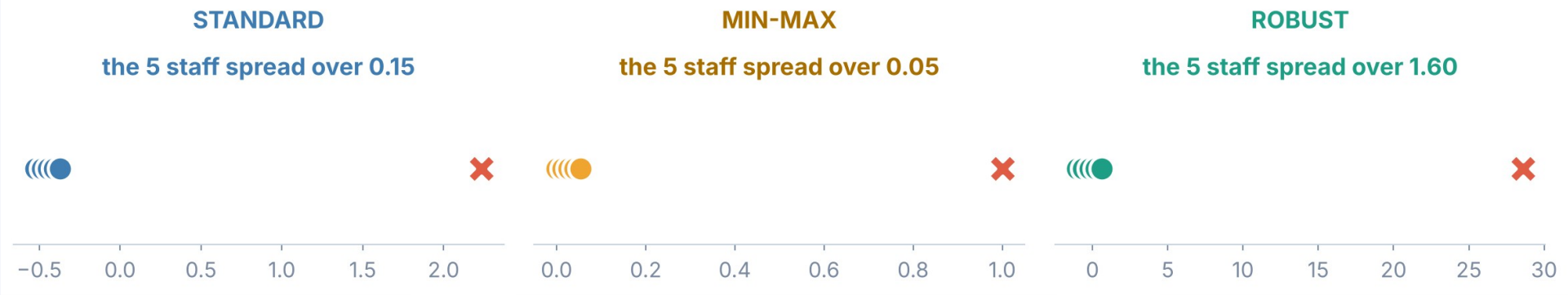
Uses the middle of the data, so one wild value cannot move it.

Same six numbers in. Three different questions answered.

2.3 · THE OUTLIER TEST

⚠️ Now watch what one CEO does to each

× is the CEO. The five dots are everybody else. Only robust keeps them apart.



0.15

STANDARD: the five staff squeezed into this much

0.05

MIN-MAX: squeezed into almost nothing

1.60

ROBUST: still properly spread out

Min-max is defined by the min and the max — the two values in your data most likely to be wrong.

2.4 · THE DECISION

So which one should I use?

STANDARD

Your normal choice.

Centres on 0, spreads to about ±2.

Use it unless you have a reason not to.

MIN-MAX

When you need 0 to 1.

Squeezes everything into [0, 1].

Fragile: one wild value ruins it.

ROBUST

When outliers are real.

Uses the median instead of the mean.

One CEO cannot move it.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()          #  your default, nine times out of ten
```

If you cannot decide, use StandardScaler. It is almost never the wrong answer.

SECTION 3 OF 5

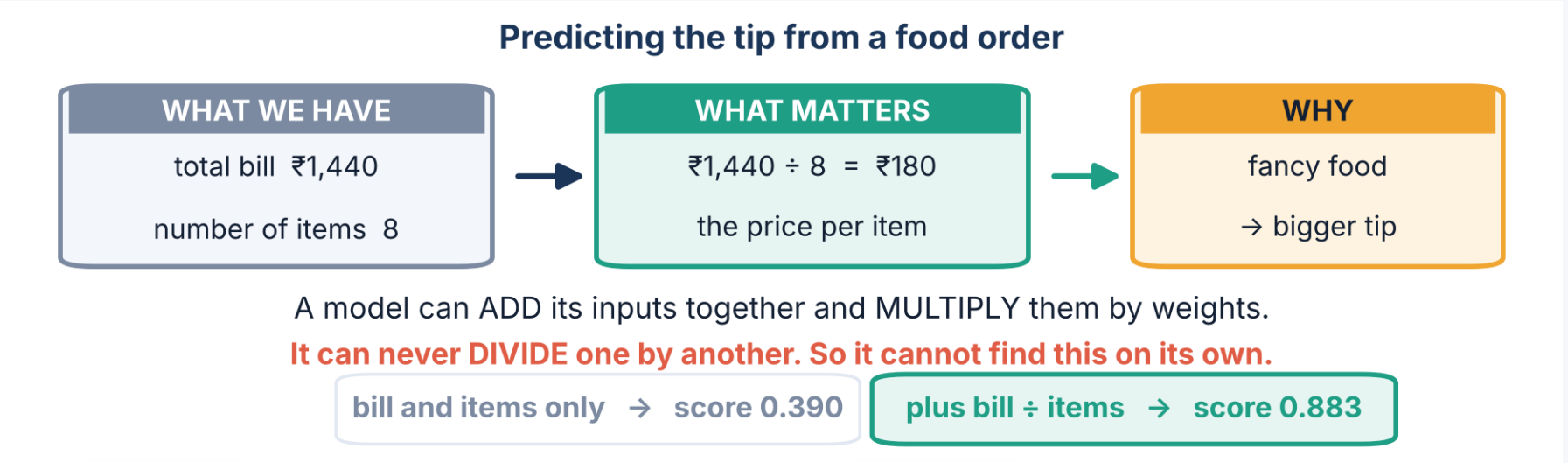
Making new features

Sometimes the column the model needs is not in your table — but you can build it out of the ones that are.

- A ratio
- A date
- Four recipes
- Measuring it

3.1 · BUILD A RATIO

A model cannot divide one column by another



0.390

with the bill and the item count

0.883

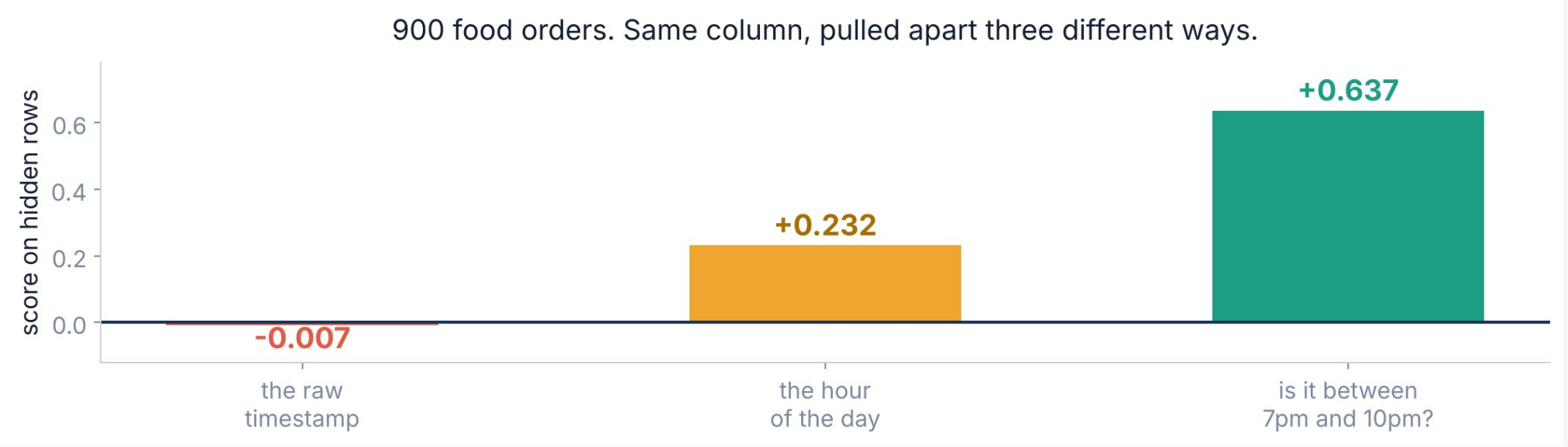
after adding bill ÷ items

No new measurement was taken. We just rearranged two columns the model already had.

3.2 · PULL A DATE APART

A date is never a feature

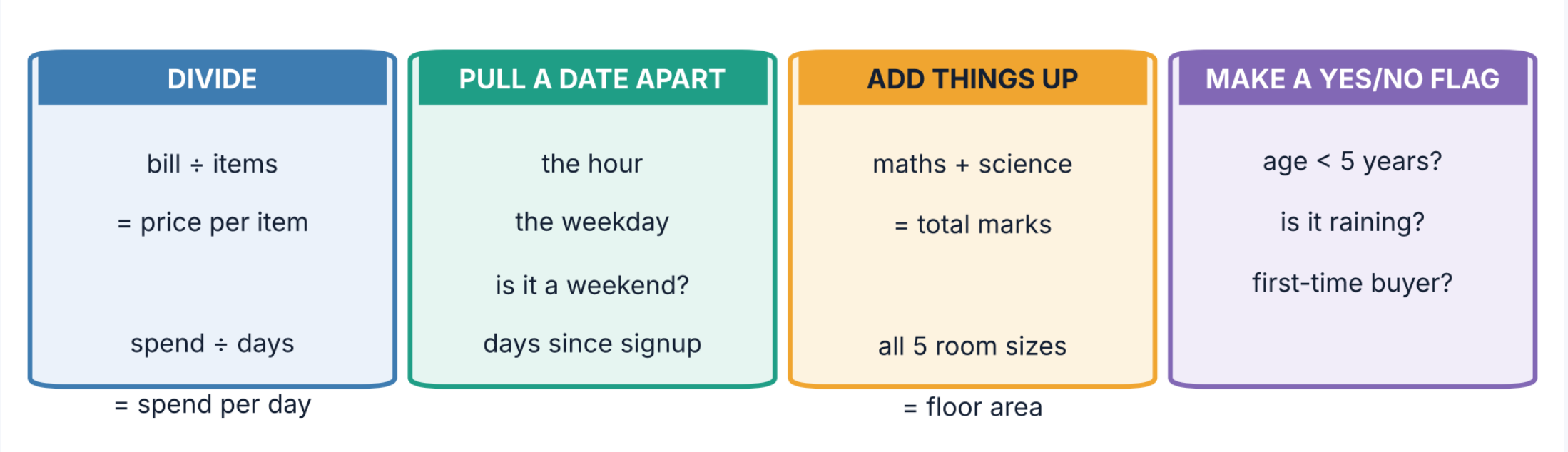
EXAMPLE **900 food orders. We want to predict how long the delivery will take.**



The timestamp as a raw number is worthless — it just counts seconds since January. Pull out the HOUR and it starts working. Ask the real question — "is it dinner time?" — and it works properly.

3.3 · THE RECIPES

Four recipes you will use again and again



Every one of these is built from columns you already have. You are not adding information — you are putting it in a shape the model can use.

⚠ **One rule** Add features ONE AT A TIME and check the score each time. A feature that does not move the score is a feature you should delete.

SECTION 4 OF 5

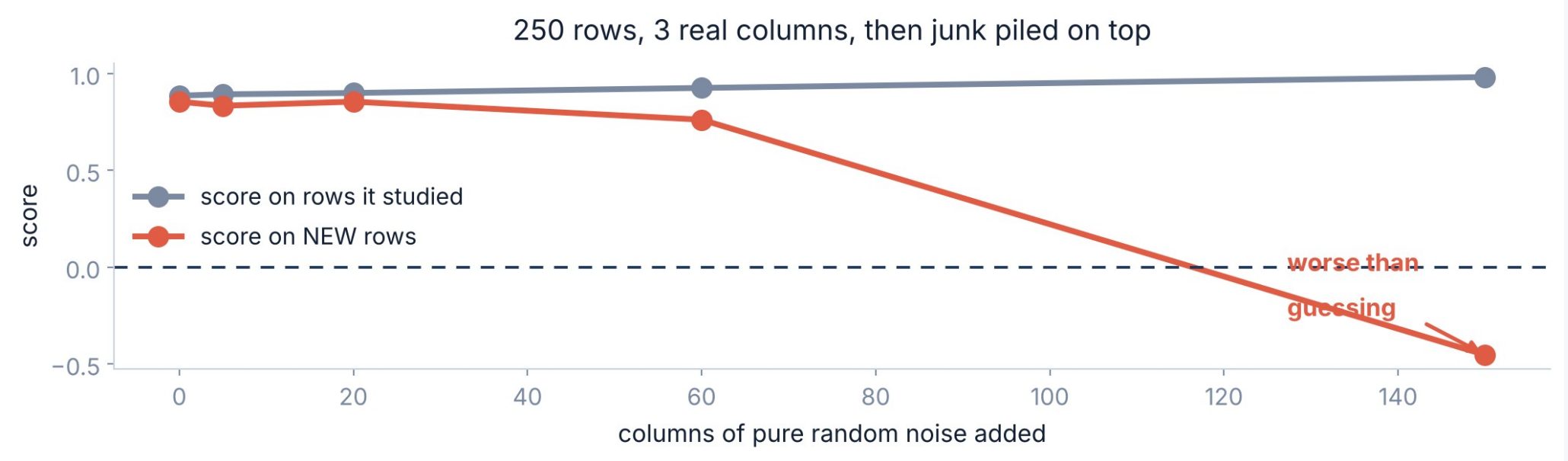
Too many features

More columns feels like more information. Past a point it is the opposite.

- Piling on junk
- Picking the good ones
- ⚠️ The cheating version

4.1 · MORE IS NOT BETTER

What happens when you keep adding columns



The grey line keeps climbing — the model looks better and better on the rows it studied. The red line falls off a cliff.

Why

With 250 rows and 153 columns, the model has nearly one knob per row. It stops finding patterns and starts remembering rows.

4.2 · PICKING FEATURES

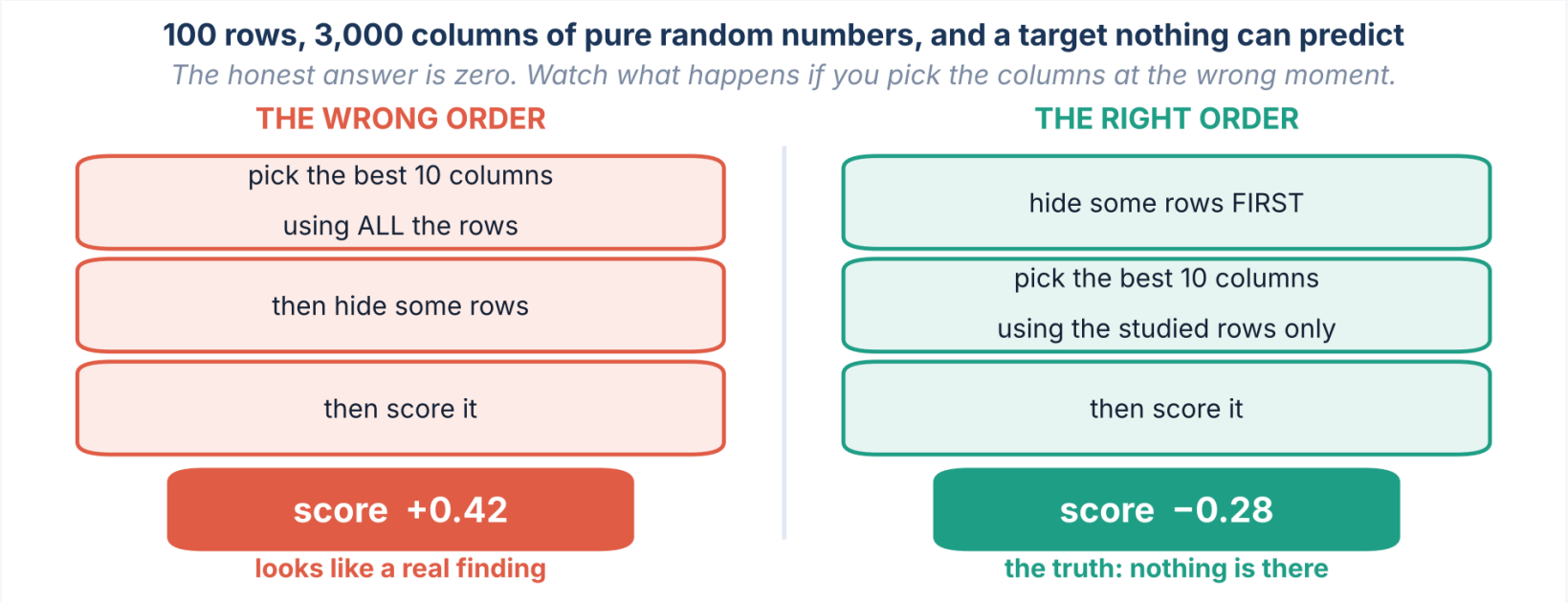
So keep only the columns that help

Score each column on its own	Keep the ones most related to the answer. Fast, and it is what we use today.	`SelectKBest`
Let the model decide	Some models shrink useless columns to zero weight all by themselves.	`Lasso, tree importances`
Try adding and removing	Fit the model again and again with different sets. Thorough, and slow.	`RFE`

Start with the first one. It is simple, it is fast, and it is right most of the time.

4.3 · THE TRAP

⚠️ But pick them at the WRONG moment and you fool yourself



There is genuinely nothing in that data. The +0.42 came entirely from letting the hidden rows help choose the columns.

Hide the rows FIRST.
Then do everything else.

Scaling, building features, picking columns — every one of them must learn only from the rows you kept.

SECTION 5 OF 5

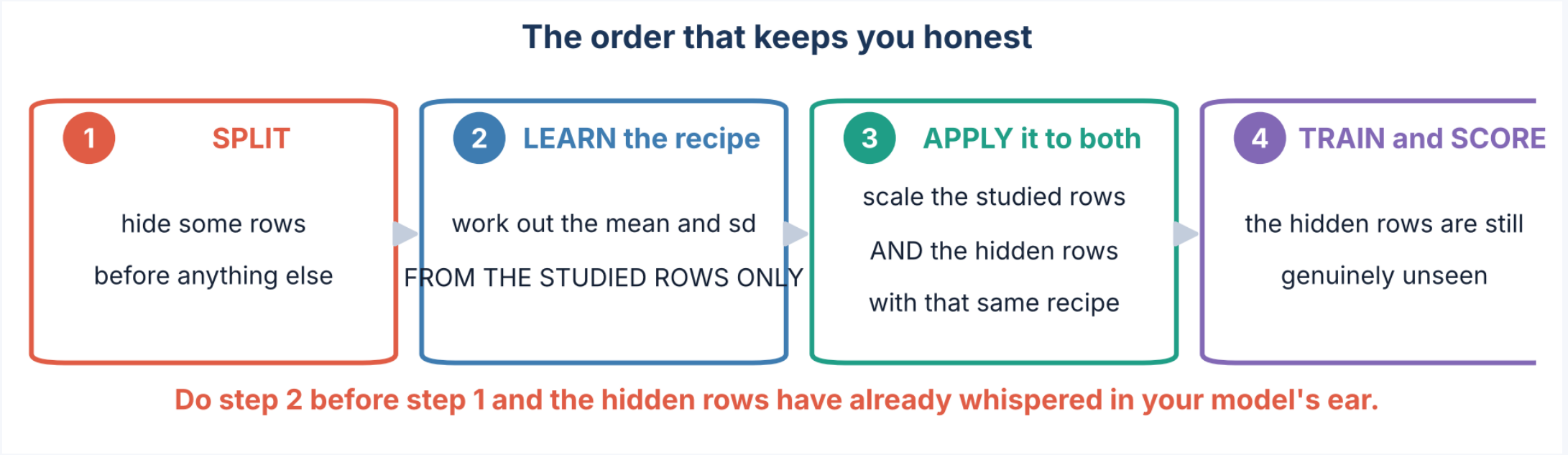
Put it together

The order to do things in, and the code to do them with.

- The right order
- The full code
- Mistakes
- Practice

5.1 · THE ORDER

The order that keeps you honest



Why it matters for SCALING

If you work out the mean using all the rows, the hidden rows have already influenced your model. The damage is usually small — we measured about 0.001 — but it is free to avoid.

Why it matters for PICKING COLUMNS

Here the damage is enormous. We just turned pure random noise into a score of +0.42 by choosing columns too early.

5.2 · THE CODE

The whole of today, in one program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor

#  STEP 2 - build the useful columns first
orders["per_item"] = orders["bill"] / orders["items"]
orders["hour"]     = orders["ordered_at"].dt.hour

X = orders.drop(columns=["tip", "ordered_at"])
y = orders["tip"]

#  STEP 1 - hide rows BEFORE anything learns from them
X_train, X_hidden, y_train, y_hidden = train_test_split(
        X, y, test_size=0.3, random_state=0)

#  STEP 3 - learn the recipe from the TRAINING rows only
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_hidden_s = scaler.transform(X_hidden)     # same recipe, not a new one

#  STEP 5 - train and score
model = KNeighborsRegressor(7).fit(X_train_s, y_train)
print(model.score(X_hidden_s, y_hidden))
```

fit vs transform

.fit() LEARNS the mean and sd. .transform() APPLIES them. Learn once, apply twice.

fit on train only

Never .fit() on the hidden rows. That is the whole point of hiding them.

Engineer before splitting?

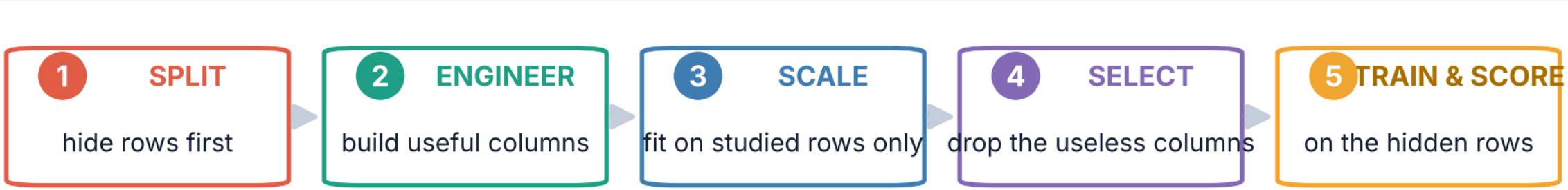
Yes — a ratio uses only that one row, so it cannot leak. Scaling uses ALL rows, so it must come after.

Trees need none of this

If you are using a random forest, you can skip the scaler entirely.

5.3 · THE ROUTINE

The five steps, in order



Step 1 comes first. Everything after it learns only from the rows you kept.

Why SPLIT comes first

Everything after it must learn from the kept rows only. Get this wrong and every number you report is inflated.

Why ENGINEER before SCALE

Build the ratio first, then scale it along with everything else. Scaling a column you are about to divide is wasted work.

Why SELECT last

You can only tell which columns help once they are all built and on a fair footing.

Write these five words down. They are the answer to most "why is my score wrong?" questions.

5.4 · DEBUG CORNER

Six mistakes everybody makes once

#	The mistake	What you will see	The fix
1	**Not scaling before kNN**	A terrible score for no obvious reason	StandardScaler first
2	**Scaling a decision tree**	Nothing. It is harmless, just pointless	Skip it for trees
3	**Min-max on data with outliers**	Every real value squashed into a corner	Use RobustScaler
4	**Fitting the scaler on everything**	A slightly inflated score, and a bad habit	Fit on train rows only
5	**Picking columns before splitting**	+0.42 on data with nothing in it	Split first, always
6	**Adding features without checking**	A model that gets worse as you "improve" it	One at a time, score each

Numbers 4 and 5 are the same mistake. Number 5 just happens to be four hundred times more expensive.

OVER TO YOU

Practice session — 30 minutes

A

Work it out

Scale six numbers by hand, three different ways

8 questions

B

Find the mistake

Four programs: wrong scaler, wrong order, wrong model

4 questions

C

Write the code

Scale, build features, select, and score honestly

5 tasks

D

Challenge

Build five features from scratch, then catch a cheating model

2 tasks

Your notebook

`Day32_Practice_Questions.ipynb`

Part A is pen and paper. The numbers are chosen to be easy.

Three habits to start today

- Split before anything else
- Fit the scaler on train rows only
- Add one feature, then check the score

Part D hands you a model scoring 0.97 and asks you to prove it is worth nothing.

Day 32 — the six things to remember

Big is not important	A price column is louder than a RAM column simply because the numbers are bigger.
Scaling changes volume, not information	kNN went from 0.056 to 0.872. Trees did not move at all.
Three scalers, one default	Standard unless you have a reason. Robust when outliers are real.
Build the column the model needs	Bill ÷ items took the score from 0.390 to 0.883 with no new data.
⚠️ More columns is not better	150 junk columns took a good model down to −0.454 on new rows.
⚠️ Split FIRST, then everything else	Picking columns too early scored +0.42 on pure random noise.

Next time — the first real model: teaching a computer to draw the best line.

Srinivasa Sai Chava · Boston University